

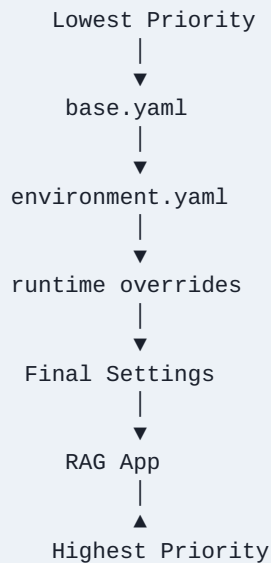
Enterprise RAG Configuration Architecture

Stage 3.4 — Configuration Overrides & Precedence

Absolutely. Let's continue directly from your working Stage 3.3 project.

Stage 3.4 — Configuration Overrides & Precedence

This stage is important because in a production application, configuration can come from multiple levels. We want to establish a clear rule:



So if the same value appears at multiple levels, the higher-priority value wins.

1. What do we have today?

Your current structure is approximately:

```
YAML-Stage3.3/
├── config/
│   ├── base.yaml
│   ├── development.yaml
│   ├── testing.yaml
│   └── production.yaml
├── src/
│   ├── __init__.py
│   ├── yaml_loader.py
│   ├── config_loader.py
│   └── settings.py
└── main.py
```

For example:

`base.yaml`

```
retrieval:
  top_k: 5
  reranking: false
```

production.yaml

```
retrieval:
  top_k: 10
```

The resulting configuration is:

```
retrieval:
  top_k: 10
  reranking: false
```

because production overrides base.
That's already one level of precedence.
Now we're going to introduce a third level.

2. Our target architecture

We'll support:

```
base.yaml
  ↓
environment.yaml
  ↓
runtime overrides
  ↓
Pydantic validation
  ↓
Settings
```

For example:

```
base.yaml
top_k = 5
  ↓
production.yaml
top_k = 10
  ↓
runtime
top_k = 20
  ↓
FINAL
top_k = 20
```

This is configuration precedence.

3. Why would runtime overrides be useful?

Imagine you're testing your RAG application.

Your production configuration says:

```
retrieval:
  top_k: 10
```

But you want to temporarily test:

```
top_k = 20
```

You don't want to modify:

```
production.yaml
```

because that's your controlled production configuration.

Instead:

```
python main.py production --top-k 20
```

The runtime value temporarily overrides the YAML value.

This is particularly useful for:

- experimentation
- A/B testing
- debugging
- container deployments
- CI/CD
- performance tuning

4. First, create a runtime override model

We'll use Pydantic for the runtime configuration as well.

File `src/settings.py`

Add this model before Settings:

```

from typing import Optional

from pydantic import BaseModel

class RuntimeOverrides(BaseModel):
    """
    Runtime configuration values that can override
    values loaded from YAML.
    """

    top_k: Optional[int] = None
    temperature: Optional[float] = None

```

Why Optional?

Because we don't necessarily want to override everything.

For example:

```

RuntimeOverrides(
    top_k=20
)

```

means:

```

top_k → override
temperature → don't change

```

5. Modify config_loader.py

We already have:

```
load_environment_config()
```

Now add a function that applies runtime overrides.

File `src/config_loader.py`

Add:

```
def apply_runtime_overrides(
    config: dict[str, Any],
    overrides: dict[str, Any],
) -> dict[str, Any]:
    """
    Apply runtime configuration overrides to the
    already merged YAML configuration.
    """

    for key, value in overrides.items():

        # Ignore None values because they mean
        # "no runtime override was supplied".
        if value is None:
            continue

        # Handle top_k override.
        if key == "top_k":
            config["retrieval"]["top_k"] = value

        # Handle temperature override.
        elif key == "temperature":
            config["llm"]["temperature"] = value

    return config
```

Notice that we're deliberately starting with only two runtime overrides:

- top_k
- temperature

Don't make this generic yet.

We're learning the architecture first.

6. Modify create_settings()

Now we connect everything together.

src/settings.py

Change:

```
def create_settings(environment: str) -> Settings:
```

to:

```
def create_settings(
    environment: str,
    runtime_overrides: RuntimeOverrides | None = None,
) -> Settings:
```

Then implement:

```

def create_settings(
    environment: str,
    runtime_overrides: RuntimeOverrides | None = None,
) -> Settings:
    """
    Load YAML configuration for the selected environment,
    apply runtime overrides, and validate the final
    configuration using Pydantic.
    """

    allowed_environments = {
        "development",
        "testing",
        "production",
    }

    # Validate environment.
    if environment not in allowed_environments:
        raise ValueError(
            f"Invalid environment '{environment}'. "
            f"Choose from: {sorted(allowed_environments)}"
        )

    # -----
    # Step 1: Load base.yaml + environment.yaml
    # -----

    config = load_environment_config(
        CONFIG_DIR,
        environment,
    )

    # -----
    # Step 2: Apply runtime overrides
    # -----

    if runtime_overrides is not None:
        config = apply_runtime_overrides(
            config,
            runtime_overrides.model_dump(),
        )

    # -----
    # Step 3: Validate final configuration
    # -----

    return Settings(**config)

```

You will also need this import at the top:

```

from src.config_loader import (
    load_environment_config,
    apply_runtime_overrides,
)

```

7. Update main.py

Now we'll allow command-line arguments.

```
main.py
```

Replace your current argument handling with:

```

import argparse

from src.settings import (
    create_settings,
    RuntimeOverrides,
)

# =====
# Create command-line argument parser
# =====

parser = argparse.ArgumentParser(
    description="Enterprise RAG application"
)

# Environment argument
parser.add_argument(
    "environment",
    choices=[
        "development",
        "testing",
        "production",
    ],
    help="Application environment",
)

# Runtime override: top_k
parser.add_argument(
    "--top-k",
    type=int,
    default=None,
    help="Override retrieval top_k",
)

# Runtime override: temperature
parser.add_argument(
    "--temperature",
    type=float,
    default=None,
    help="Override LLM temperature",
)

# Parse command-line arguments
args = parser.parse_args()

# =====
# Create runtime override object
# =====

runtime_overrides = RuntimeOverrides(
    top_k=args.top_k,
    temperature=args.temperature,

```



```

)

# =====
# Create final Settings object
# =====

settings = create_settings(
    args.environment,
    runtime_overrides,
)

# =====
# Display configuration
# =====

print("=====")
print("Enterprise RAG Configuration")
print("=====")

print("Environment:", settings.application.environment)

print()
print("LLM:")
print("Provider:", settings.llm.provider)
print("Model:", settings.llm.model)
print("Temperature:", settings.llm.temperature)

print()
print("Retrieval:")
print("Top K:", settings.retrieval.top_k)
print("Reranking:", settings.retrieval.reranking)

```

8. Test without an override

Run:

```
python main.py production
```

Your production.yaml contains:

```
retrieval:
  top_k: 10
```

Therefore:

```
Top K: 10
```

9. Now override it

Run:

```
python main.py production --top-k 20
```

Now:

```
Top K: 20
```

What happened?

```
base.yaml
top_k = 5
  ↓
production.yaml
top_k = 10
  ↓
--top-k 20
top_k = 20
  ↓
Pydantic
  ↓
Settings
```

So:

Runtime override wins over environment configuration.

10. Test temperature

Suppose base.yaml has:

```
llm:
  temperature: 0
```

Run:

```
python main.py production --temperature 0.7
```

You should get:

```
Temperature: 0.7
```

Again:

```
YAML = 0
Runtime = 0.7

Final = 0.7
```

11. Test both together

You can override both:

```
python main.py production --top-k 20 --temperature 0.7
```

Result:

Environment: production

LLM:

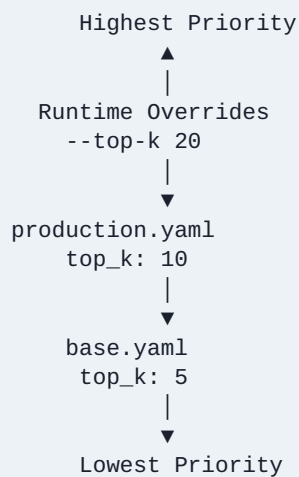
Temperature: 0.7

Retrieval:

Top K: 20

12. The complete precedence model

You've now implemented:



Therefore:

Runtime > Environment YAML > Base YAML

This is the central concept of Stage 3.4.

13. One important improvement

There's currently a slight architectural weakness in:

```
apply_runtime_overrides()
```

because we're doing:

```
if key == "top_k":  
    config["retrieval"]["top_k"] = value
```

and:

```
if key == "temperature":
    config["llm"]["temperature"] = value
```

As your application grows, this would become ugly:

```
if key == "...":
    ...
if key == "...":
    ...
if key == "...":
    ...
```

We don't want that in our final Enterprise RAG architecture.

Instead, our eventual design should allow runtime configuration to have the same structure as YAML:

```
{
  "llm": {
    "temperature": 0.7
  },
  "retrieval": {
    "top_k": 20
  }
}
```

Then we can use the same recursive merge mechanism we already built.

That's the next refinement.

14. Better architecture — reuse our existing merge function

We already have:

```
merge_configs()
```

It recursively merges nested dictionaries.

So instead of creating special logic for:

- top_k
- temperature

we can convert runtime overrides into:

```
runtime_config = {
  "llm": {
    "temperature": 0.7
  },
  "retrieval": {
    "top_k": 20
  }
}
```

and simply do:

```
merge_configs(  
    config,  
    runtime_config,  
)
```

Now our architecture becomes beautifully consistent:

```
base.yaml  
  ↓  
environment.yaml  
  ↓  
runtime configuration  
  ↓  
same merge mechanism  
  ↓  
Pydantic  
  ↓  
Settings
```

That is the direction I'd recommend for your Enterprise RAG configuration architecture.

15. Why this matters for GenAI and Agentic AI

Think about your future RAG system.

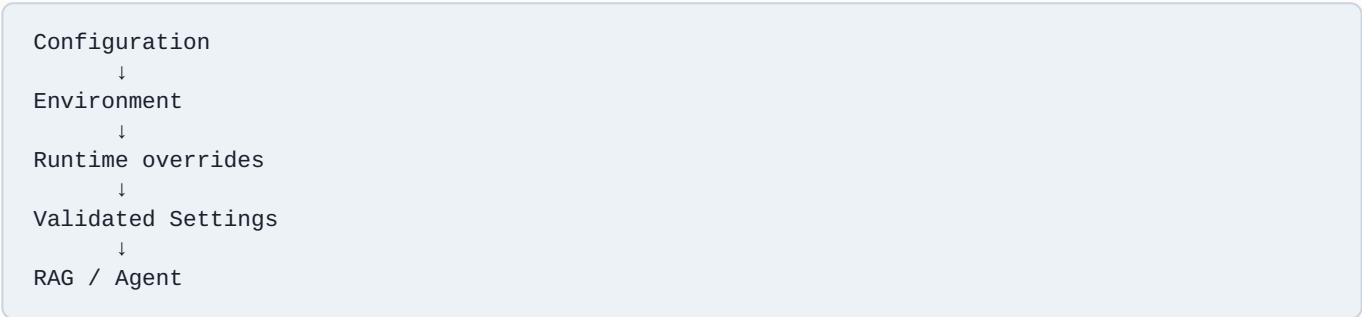
You might have:

```
llm:  
  model: llama-3.1-8b-instant  
  temperature: 0  
  
retrieval:  
  top_k: 5  
  
reranking:  
  enabled: true  
  
chunking:  
  strategy: recursive  
  chunk_size: 1000  
  
agent:  
  max_iterations: 10
```

During production experimentation, you might want:

```
top_k = 10  
chunk_size = 1500  
max_iterations = 15
```

You don't want to edit the source code.
You want:



This is exactly the kind of configuration-driven architecture that makes GenAI systems easier to operate and evolve.

Your Stage 3 journey now

Stage 3	YAML Application Configuration	✓
Stage 3.1	Settings Object	✓
Stage 3.2	Pydantic Validation	✓
Stage 3.3	Environment Configuration	✓
Stage 3.4	Configuration Precedence	↺

And the precedence rule we've established is:

Runtime > Environment > Base

The next refinement I recommend is to replace our hardcoded top_k/temperature override logic with a generic recursive merge, so the same mechanism can handle any nested RAG/Agent configuration without modifying config_loader.py every time we introduce a new setting.